

02 · Bag-of-Words

Parte 2 — representaciones dispersas. La primera forma de vectorizar texto es la más literal posible: contar cuántas veces aparece cada palabra. Se trabaja primero sobre el corpus de juguete (01_fundamentos...ipynb) y luego sobre los comentarios reales de clientes de la tienda virtual.

```
corpus = {  
    "D1": "el gato duerme en el sofá",  
    "D2": "el perro duerme en la alfombra",  
    "D3": "el sismo sacudió la costa",  
}
```

Paso 1 — construir el vocabulario

Se juntan todas las palabras distintas del corpus y se ordenan. Ese orden ya no cambia: define las posiciones del vector.

```
vocabulario = sorted(set(" ".join(corpus.values()).split()))  
print(f"Vocabulario V ({len(vocabulario)} palabras distintas):")  
for posicion, palabra in enumerate(vocabulario):  
    print(f" {posicion:>2} {palabra}")
```

```
Vocabulario V (11 palabras distintas):  
 0 alfombra  
 1 costa  
 2 duerme  
 3 el  
 4 en  
 5 gato  
 6 la  
 7 perro  
 8 sacudió  
 9 sismo  
10 sofá
```

En un corpus de noticias real el vocabulario no tendría 11 entradas sino unas 50 000. La escala del problema aparece más abajo, con datos reales.

Paso 2 — matriz de conteos

`CountVectorizer` hace exactamente esto: fija un vocabulario y cuenta, por documento, cuántas veces aparece cada palabra.

```
import pandas as pd  
from sklearn.feature_extraction.text import CountVectorizer  
  
count_vectorizer = CountVectorizer(token_pattern=r"(?u)\b\w+\b")  
X_counts = count_vectorizer.fit_transform(corpus.values())
```

```
matriz_bow = pd.DataFrame(
    X_counts.toarray(),
    columns=count_vectorizer.get_feature_names_out(),
    index=corpus.keys(),
)
matriz_bow
```

	alfombra	costa	duerme	el	en	gato	la	perro	sacudió	sismo	sofá
D1	0	0	1	2	1	1	0	0	0	0	1
D2	1	0	1	1	1	0	1	1	0	0	0
D3	0	1	0	1	0	0	1	0	1	1	0

En D1 la palabra "el" aparece dos veces, y por eso su columna marca 2. Cada frase es ahora una lista de 11 números: ya se pueden calcular cosenos.

Limitación: casi todo son ceros. El nombre "bolsa de palabras" alude a que el orden no se conserva — cuánto pesa esto se mide más abajo, sobre el corpus real.

Similitud entre documentos con Bag-of-Words

```
import numpy as np

def similitud_coseno(a, b):
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    return a.dot(b) / (np.linalg.norm(a) * np.linalg.norm(b))

cos_d1_d2 = similitud_coseno(matriz_bow.loc["D1"], matriz_bow.loc["D2"])
cos_d1_d3 = similitud_coseno(matriz_bow.loc["D1"], matriz_bow.loc["D3"])
print(f"cos(D1, D2) = {cos_d1_d2:.3f} (gato duerme vs perro duerme)")
print(f"cos(D1, D3) = {cos_d1_d3:.3f} (gato duerme vs sismo costa)")

cos(D1, D2) = 0.577 (gato duerme vs perro duerme)
cos(D1, D3) = 0.316 (gato duerme vs sismo costa)
```

El orden es correcto (D1-D2 > D1-D3), pero 0.316 resulta excesivo para dos frases sin ninguna relación temática: se debe a que ambas contienen "el". Corresponde reducir el peso de palabras tan comunes — el motivo de ser de TF-IDF, en el siguiente notebook.

Limitación: la pérdida del orden

Bag-of-Words no distingue sujeto de objeto: solo importa qué palabras aparecen, no en qué posición.

```
frases_opuestas = ["El perro mordió al hombre", "El hombre mordió al perro"]
X_opuestas = CountVectorizer(token_pattern=r"(?u)\b\w+\b",
    lowercase=True).fit_transform(frases_opuestas)

print("Vectores idénticos:", (X_opuestas[0].toarray() ==
    X_opuestas[1].toarray()).all())
```

```
print(f"cos(frase 1, frase 2) = {similitud_coseno(X_opuestas[0].toarray()[0],
X_opuestas[1].toarray()[0]):.3f}")
```

Vectores idénticos: True
cos(frase 1, frase 2) = 1.000

Para el modelo, ambas frases son idénticas aunque signifiquen lo contrario. Contar pares de palabras (bigramas) recupera algo de orden, pero dispara el vocabulario — se explora el costo real en el ejercicio de `03_tfidf.ipynb`. El orden se recupera de verdad recién con los Transformers (`05_...ipynb`).

Aplicación al corpus real: comentarios de clientes

Antes de vectorizar hace falta normalizar el texto: minúsculas, sin puntuación, sin *stopwords* y lematizado (para que "duermo" y "durmiendo" cuenten como la misma palabra). Se usa spaCy en español.

```
import spacy

nlp = spacy.load("es_core_news_sm")

def normalizar(texto):
    doc = nlp(str(texto).lower())
    lemas = [token.lemma_ for token in doc if token.is_alpha and not token.is_stop]
    return " ".join(lemas)

def normalizar_muchos(textos):
    return [
        " ".join(t.lemma_ for t in doc if t.is_alpha and not t.is_stop)
        for doc in nlp.pipe((str(x).lower() for x in textos), batch_size=64)
    ]
```

```
comentarios = pd.read_csv("../data/comentarios.csv")
comentarios["texto_normalizado"] = normalizar_muchos(comentarios["texto_comentario"])
comentarios[["texto_comentario", "texto_normalizado"]].head(3)
```

	texto_comentario	texto_normalizado
0	El Smartphone Nexus 5G es un cambio de juego. ...	smartphonir nexus g cambio juego pantalla oled...
1	La Camiseta Deportiva Ultralight es muy cómoda...	camiseta deportivo ultralight cómodo entrenami...
2	La Laptop Gamer Pro es una bestia de rendimien...	laptop gamer pro bestia rendimiento superar ex...

```
cv_real = CountVectorizer(min_df=2)
X_real = cv_real.fit_transform(comentarios["texto_normalizado"])

n_docs, n_vocab = X_real.shape
dispersidad = 1 - X_real.nnz / (n_docs * n_vocab)
print(f"Matriz de conteos: {n_docs} comentarios x {n_vocab} palabras")
print(f"Dispersidad (proporción de ceros): {dispersidad:.1%}")
```

Matriz de conteos: 620 comentarios x 514 palabras
Dispersidad (proporción de ceros): 97.9%

Con 619 comentarios reales el vocabulario ya crece a varios cientos de palabras y la matriz es abrumadoramente cero — la limitación de la slide teórica ("con 50 000 palabras, 99.9% de cada fila sería cero") se confirma aquí con datos propios, aunque a menor escala.

Siguiente: `03_tfidf.ipynb` — pesar cada palabra por su rareza en el corpus, y el Ejercicio 01 completo sobre este mismo corpus de comentarios.